Podstawy programowania w Java

mgr inż. Sławomir Aleksander Dańczak

Katedra Metod Programowania
Polsko-Japońska Akademia Technik Komputerowych



JAVA

Konwersja, operator wyboru, instrukcja warunkowa i decyzyjna

Wykład 03

Plan na dzisiaj...

00

OPERATOR KONWERSJI → OPERATOR WYBORU → INSTRUKCJA WARUNKOWA → INSTRUKCJA DECYZYJNA

Operator konwersji

01

Zmiana sposobu interpretacji wartości

WARTOŚĆ → TYP ŹRÓDŁOWY → KONWERSJA → TYP DOCELOWY

Operator konwersji

Czym jest?

(TYP)wartość

Konwersja typów to proces zmiany typu danych z jednego na inny, aby umożliwić operacje, przypisania lub wywołania metod.

Zapewnienie kompatybilności typów, np. podczas przypisywania wartości, arytmetyki lub dziedziczenia klas.

Rodzaje konwersji typów prostych

- rozszerzająca (Widening Conversion)
- zawężająca (Narrowing Conversion) - wymaga jawnego rzutowania, ponieważ może powodować utratę danych lub precyzji.

Operator wyboru

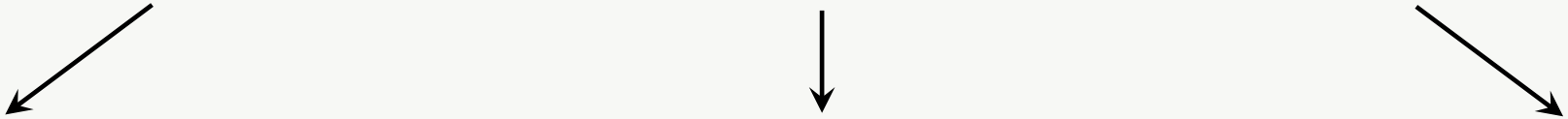
02

Wartość na podstawie warunku

WARUNEK → TRUE/FALSE → WYBÓR → WARTOŚĆ

Operator wyboru

```
warunek_logiczny ? wartość_gdy_prawda : wartość_gdy_fałsz
```



wartość logiczna — wartość zwracana gdy true — wartość zwracana gdy false

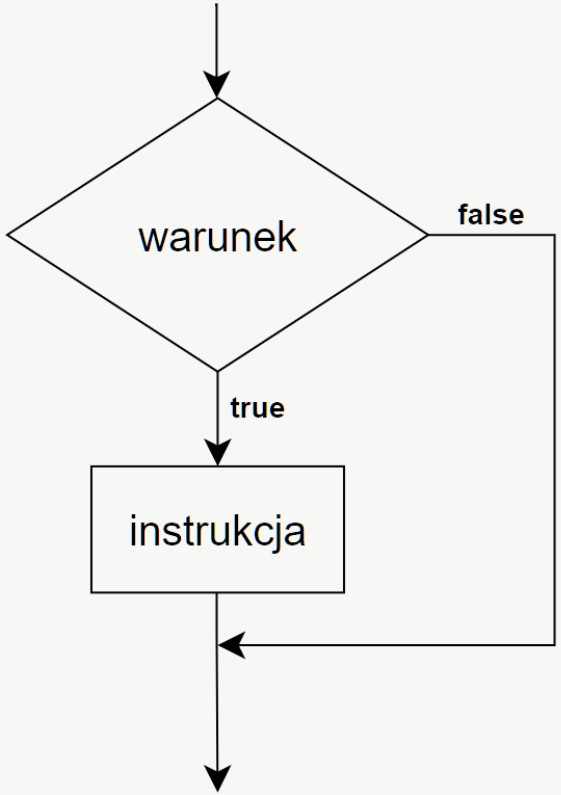
Instrukcja warunkowa 03

Sterowanie wykonaniem programu za pomocą warunku

WARUNEK → TRUE/FALSE → WYBÓR ŚCIEŻKI → WYKONANIE

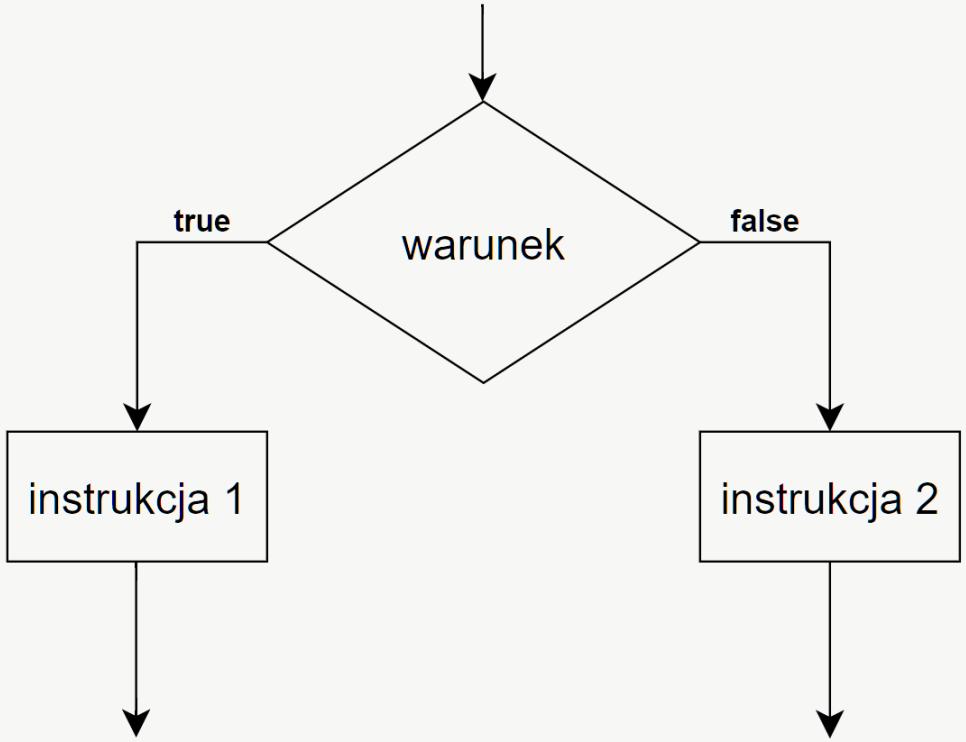
Instrukcja warunkowa

```
if (warunek_logiczny)
    instrukcja;
```



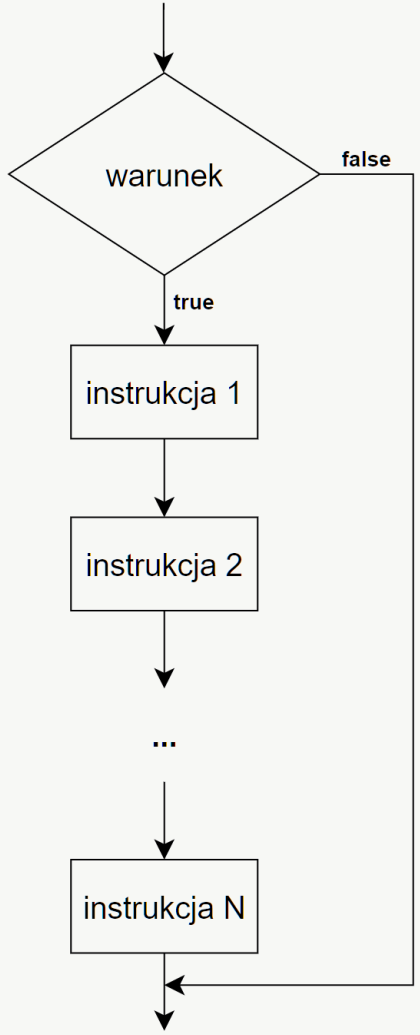
Instrukcja warunkowa

```
if (warunek_logiczny)
    instrukcja1;
else
    instrukcja2;
```



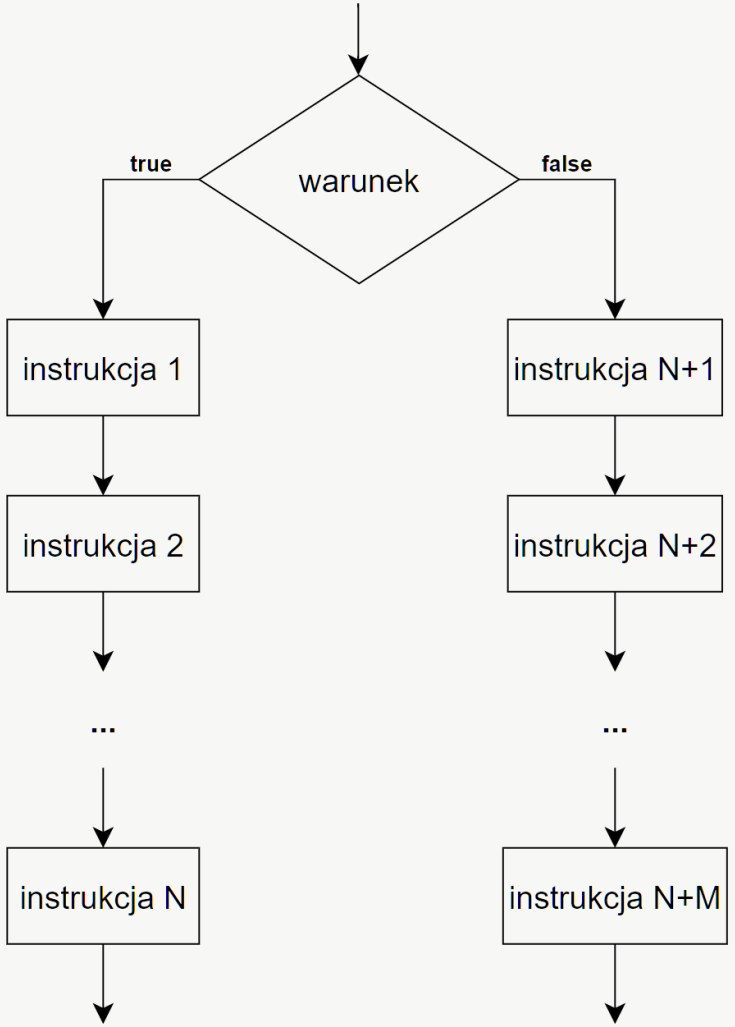
Instrukcja warunkowa

```
if (warunek_logiczny) {
    instrukcja1;
    instrukcja2;
    ...
    instrukcjaN;
}
```



Instrukcja warunkowa

```
if (warunek_logiczny) {
    instrukcja1;
    instrukcja2;
    ...
    instrukcjaN;
} else {
    instrukcjaN+1;
    instrukcjaN+2;
    ...
    instrukcjaN+M;
}
```



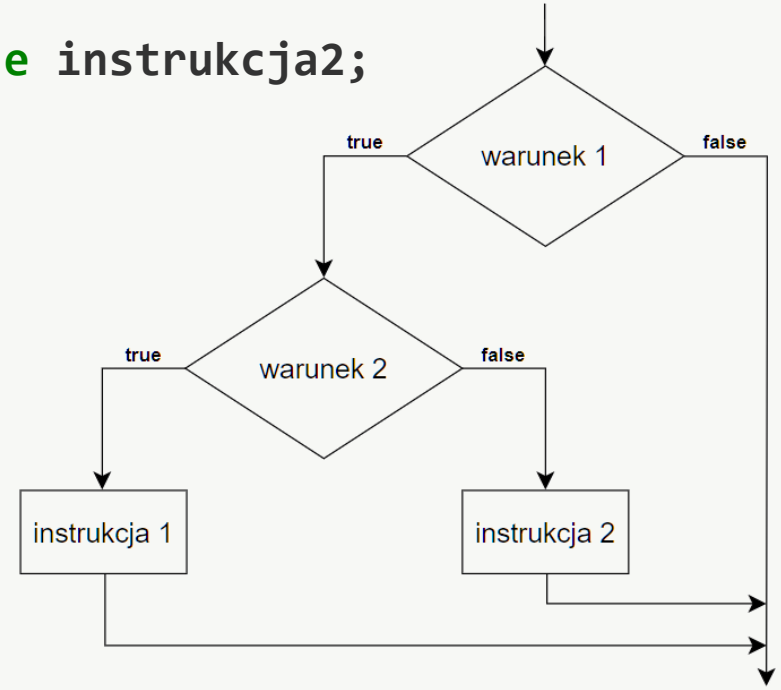
Instrukcja warunkowa

Jeśli w instrukcji warunkowej znajduje się kolejna instrukcja warunkowa, to stosuje się zasadę, że każdej fazie **else** odpowiada najbliższa poprzedzająca ją faza **if** (chyba, że nawiasy grupujące zmieniają kolejność).

```
if (warunek1) if (warunek2) instrukcja1; else instrukcja2;
```

jest równoważne:

```
if(warunek1){
    if(warunek2){
        instrukcja1;
    } else {
        instrukcja2;
    }
}
```



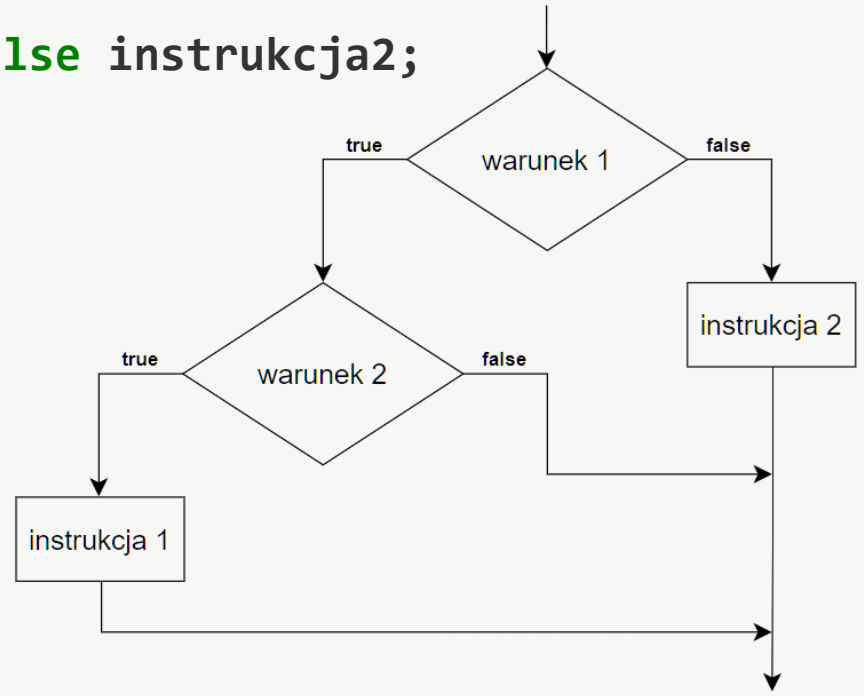
Instrukcja warunkowa

Jeśli w instrukcji warunkowej znajduje się kolejna instrukcja warunkowa, to stosuje się zasadę, że każdej fazie **else** odpowiada najbliższa poprzedzająca ją faza **if** (chyba, że nawiasy grupujące zmieniają kolejność).

```
if (warunek1) { if (warunek2) instrukcja1; } else instrukcja2;
```

jest równoważne:

```
if(warunek1){
    if(warunek2){
        instrukcja1;
    }
} else {
    instrukcja2;
}
```



Instrukcja decyzyjna 04

Wybór działania na podstawie wartości

WARTOŚĆ → DOPASOWANIE → WYKONANIE

Instrukcja decyzyjna

Instrukcja decyzyjna jest używana do uproszczenia decyzji w programie, szczególnie wtedy, gdy trzeba wykonać różne akcje względem wartości zmiennej

Instrukcja decyzyjna ma postać:

```
switch (exp) {
    case exp1 -> instrukcja1;
                 break;
    case exp2 -> instrukcja2;
                 ...
                 instrukcjaN;
                 break;
    ...
    default -> instrukcjaM;
}
```

Instrukcja decyzyjna

```
switch(day){
    case "Monday":
                    System.out.println("Start of the week");
                    break;
    case "Friday":
                    System.out.println("End of the week");
                    break;
    case "Saturday":
                    System.out.println("Weekend");
                    break;
    case "Sunday":
                    System.out.println("Weekend");
                    break;
    default: System.out.println("Midweek");
}
```

Instrukcja decyzyjna

W **Java 5** wprowadzono typ wyliczeniowy umożliwiając wybór przez podanie stanu

```
public class Main {
    public enum Day { SUNNY, CLOUDY }

    static void main() {

        Day day2 = Day.SUNNY;
        switch (day2) {
            case SUNNY:
                break;
            case CLOUDY:
                break;
            default:
                System.out.println();
        }
}
```

Instrukcja decyzyjna

W **Java 7** wprowadzono zmienną typu String jako wyróżnik stanu

```
String animal = "CAT";
String result;

switch (animal) {
    case "DOG":
                result = "domestic animal";
                break;
    case "CAT":
                result = "feline";
                break;
}
```

Instrukcja decyzyjna

W **Java 13/14** wprowadzono możliwość zwracania wartości

```
String res = switch(animal) {
    case "DOG":
           yield "it is dog";
    case "CAT":
           yield "it is cat";
    default:
           yield "unknown";
};
```

```
String res = switch(animal) {
    case "DOG", "WOLF":
           yield "it is dog";
    case "CAT", "TIGER":
           yield "it is cat";
    default:
           yield "unknown";
};
```

Instrukcja decyzyjna

W **Java 13/14** wprowadzono **switch rule** i operator **arrow**

```
String res = switch(animal) {
    case "DOG", "WOLF" -> "it is dog";
    case "CAT", "TIGER" -> "it is cat";
    default -> "unknown";
};
```

```
switch (animal) {
    case "CAT" -> System.out.println("cat");
    case "DOG" -> System.out.println("dog");
}
```

Instrukcja decyzyjna

W **Java 17** wprowadzono literał **null** jako dopuszczalny stan

```
switch (animal) {
    case "CAT" -> System.out.println("cat");
    case null -> System.out.println("it is null");
    default -> System.out.println("N/D");
}
```

Instrukcja decyzyjna

W Java 21 (w Java 17 preview) wprowadzono **parowanie wzorców (pattern matching)**

```
Object obj = "asd";
String res = switch(obj) {
    case Integer i -> "it is an integer";
    case String s -> "it is a String";
    default -> "unknown";
};
```

W Java 21 wprowadzono **dozorowanie wzorców (guarded patterns)**

```
String res5 = switch(obj) {
    case Integer i -> "it is an integer";
    case String s when s.length()>10 -> "it is a String";
    default -> "unknown";
};
```

Instrukcja decyzyjna

W **Java 21** wprowadzono możliwość używania **wzorców rekordów (Record Patterns)**

```
record Point(int x, int y) {}

Point p = new Point(10, 20);

String res6 = switch (p) {
    case Point(int x, int y) -> "x = " + x + ", y = " + y;
};
```

Instrukcja decyzyjna

Java 25 (3rd preview), Java 26 (4th preview)

Rozszerzenie pattern matching dla typów prostych

```
short a = 5;

switch (a) {
    case int val when val == 5 -> System.out.println("int");
    case short val when val == 5 -> System.out.println("short");
    default -> System.out.println("default");
}
```

Zadania wykładowe

05

01 Napisz program, który przeliczy podaną temperaturę w stopniach Celsjusza na stopnie Fahrenheita. Wynik wyświetl jako liczbę całkowitą, wykorzystując jawne rzutowanie typu. Sprawdź, co dzieje się z częścią ułamkową wyniku

02 Napisz program, który na podstawie trzech liczb całkowitych wyznaczy największą z nich. Wykorzystaj instrukcje **if-else**, a następnie spróbuj zrobić to samo za pomocą operatora **?:**

03 Napisz program symulujący prosty kalkulator. Na podstawie dwóch liczb oraz symbolu działania (+, -, *, /) wykonaj odpowiednią operację, wykorzystując instrukcję switch. Uwzględnij dzielenie przez zero oraz nieznany operator.

Pytania?

06

> waiting for input...

Dziękuję za uwagę

07

THE END